

ФІНАЛЬНИЙ ПРОЄКТ · GOIT NEOVERSITY

BrewGo

Застосунок для замовлення кави на самовивіз: меню, налаштування напою, оплата і час, коли забрати — без черги.

Ігор Яцишин · React Native · Expo SDK 57

ІДЕЯ

Кава без черги

Проблема

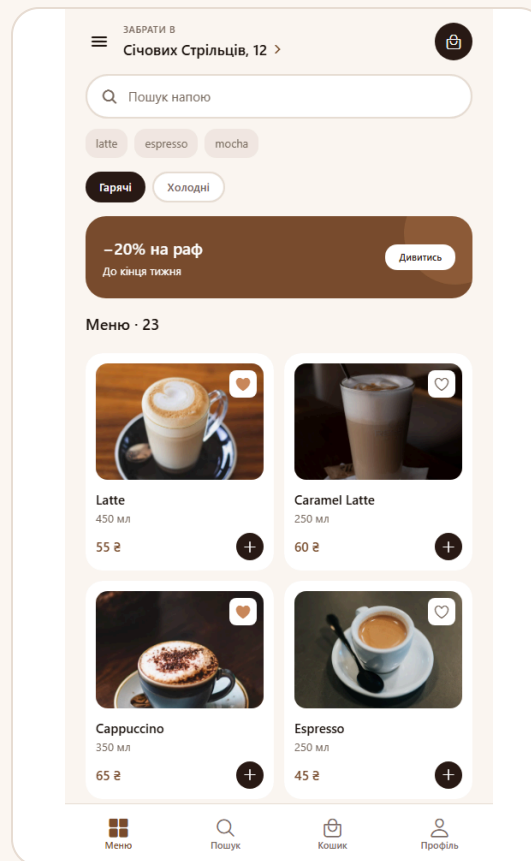
Ранкова черга в кав'ярні коштує 10–15 хвилин. Напій готується за три, решта — очікування.

Рішення

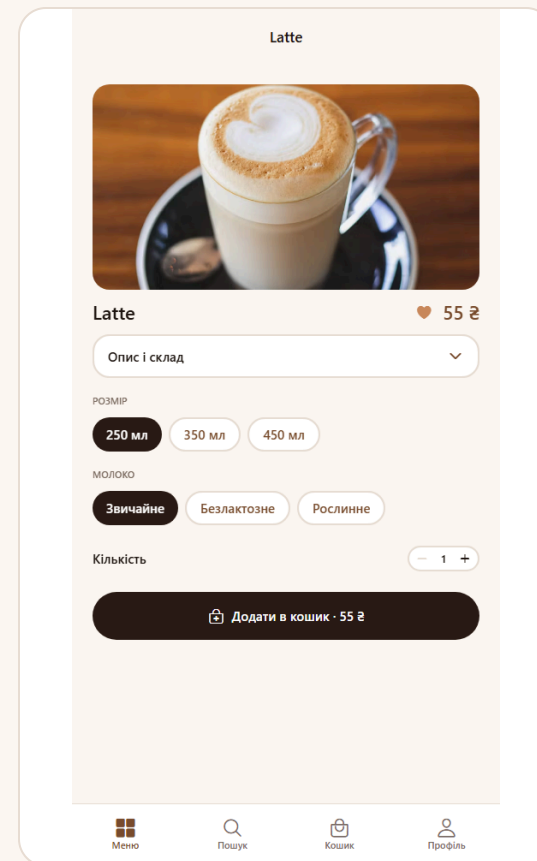
Замовлення і оплата в застосунку, час самовивозу обирає користувач. Приходиш — напій готовий.

Сценарій

Меню → налаштування напою → кошик → час і оплата → підтвердження з номером → забрати.



Меню



Напій

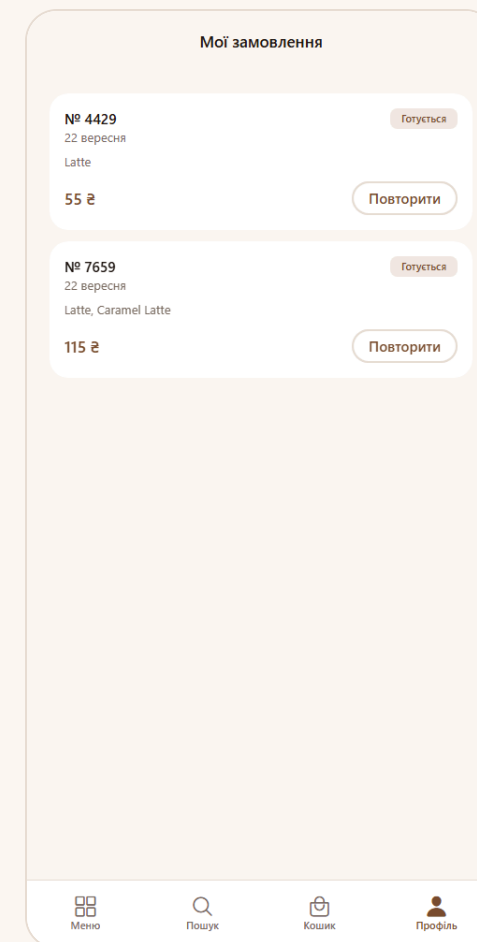
Що працювало, а що ні

ЗОНА	БУЛО	СТАЛО
Стан застосунку	Кошик і тема жили в пам'яті — перезапуск усе скидав	Кошик, замовлення, улюблене й тема зберігаються
Замовлення	Історія — захардкоджений масив, оформлене замовлення зникало	Справжні замовлення в Redux, деталі й повтор
Профіль	«6 з 8 стаканів» — фіксований текст	Бонусна картка рахує реальні стакани
Адреса в шапці	Шеврон є, натискання нічого не робить	Відкриває екран закладу з картою
Улюблене	Не існувало	Сердечко на картці, окремий екран

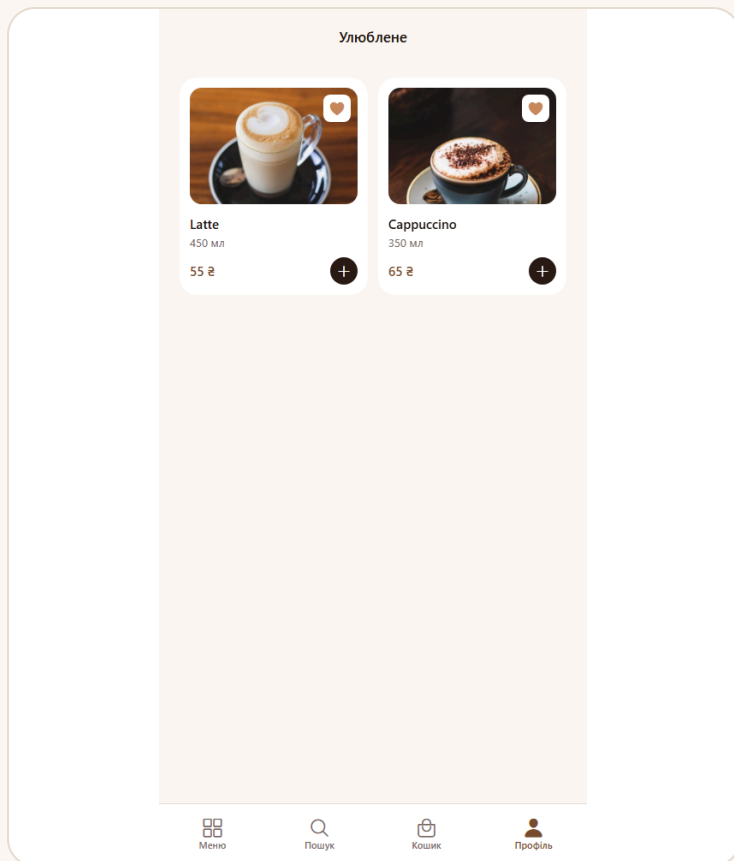
Замовлення стали справжніми

- **Оформлення створює замовлення** — позиції копіюються з кошика, кошик очищається
- **Копія, а не посилання** — історія показує, що саме куплено, навіть якщо меню й ціни зміняться
- **Деталі в модальному вікні** — склад, час, оплата, сума
- **Повтор одним натисканням** — позиції повертаються в кошик без запиту до API
- **Бонусна картка** — рахує стакани з оформлених замовлень

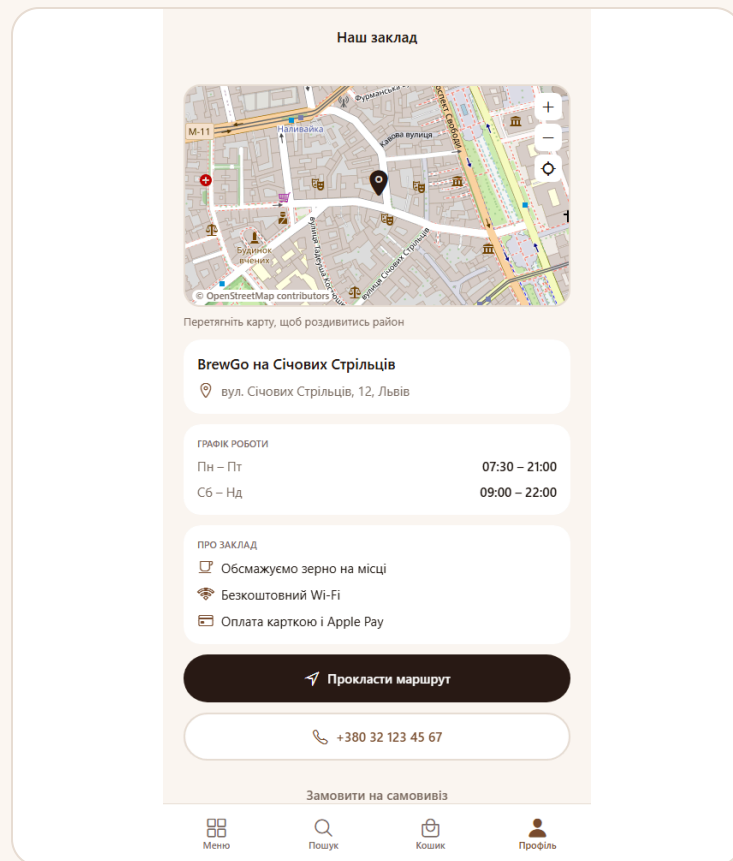
```
navigation.navigate(SCREENS.CONFIRMATION, { orderId })  
// між екранами подорожує ідентифікатор,  
// дані читаються зі сторя
```



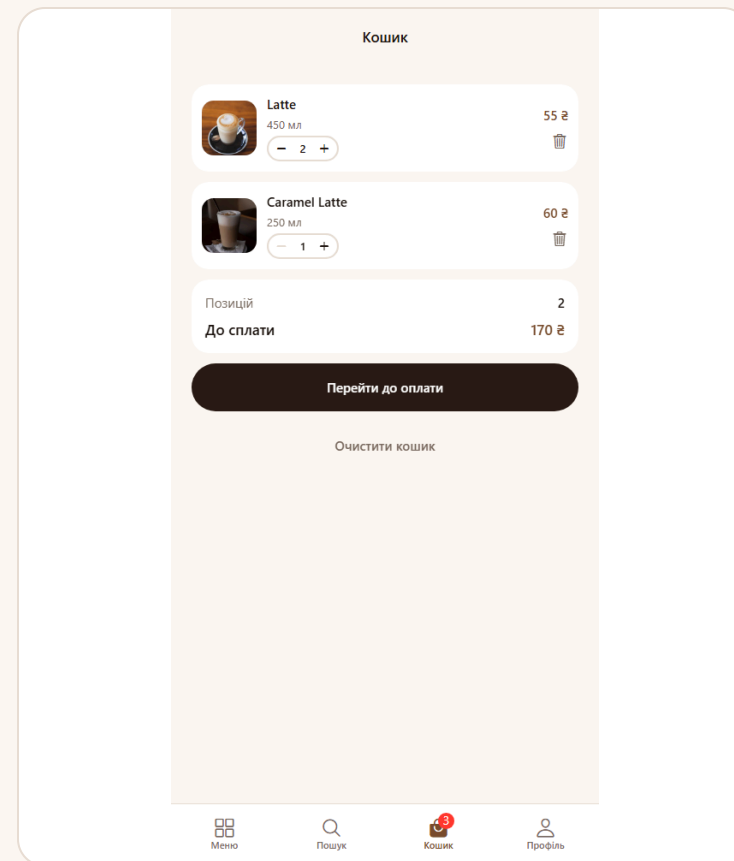
Улюблене, заклад, карта



Улюблене зберігає лише ідентифікатори



Заклад: графік, маршрут, дзвінок



Кошик із сумою та лічильником

Де Context, а де Redux

АСПЕКТ	ІНСТРУМЕНТ	ОБҐРУНТУВАННЯ
Тема	Context API	Читається майже кожним компонентом, змінюється одним перемикачем. Редюсери були б церемонією
Кошик	Redux Toolkit	Змінюється з чотирьох місць, п'ять операцій, похідні значення — лічильник і сума
Замовлення	Redux Toolkit	Створюються на оформленні, читаються на трьох екранах, дають лічильник стаканів
Улюблене	Redux Toolkit	Змінюється з двох екранів, читається з трьох, зберігається тим самим механізмом
Меню з API	локальний стан	Свідомо не в глобальному стані: залежить від категорії й потрібне двом екранам

Сорок рядків замість залежності

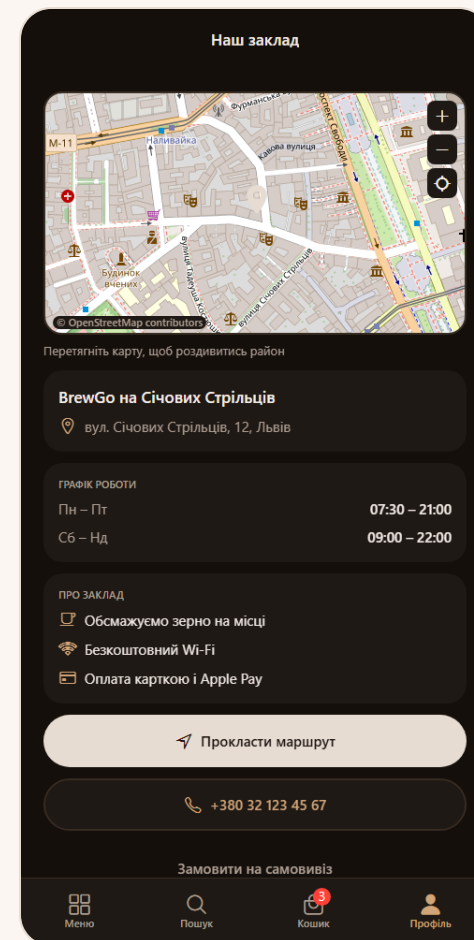
- **Без `redux-persist`** — додавати залежність одразу після роботи зі зменшення бандла було б непослідовно
- **Одна дія гідратації** — кожен слайс розбирає її сам, код завантаження не змінюється при додаванні нового
- **Дебаунс 300 мс** — степер кількості інакше писав би на диск кілька разів на секунду
- **Читання до першого кадру** — застосунок не блимає порожнім кошиком

<code>store/hydrate.js</code>	одна дія на весь застосунок
<code>store/persistMiddleware</code>	пише зріз після кожної дії
<code>storage/persistence.js</code>	JSON, дебаунс, ключі з версією
<code>hooks/useBootstrap.js</code>	читає сховище до рендера

```
extraReducers: (builder) => {  
  builder.addCase(hydrate, (state, action) => {  
    const saved = action.payload?.orders;  
    if (saved?.items) state.items = saved.items;  
  });  
}
```

Карта без картографічної залежності

- **react-native-maps не має web-реалізації** — а застосунок має працювати і в браузері
- **Растрові тайли OpenStreetMap** — сітка 5×5, проєкція Web Mercator у `utils/tiles.js`
- **Жест Pan на Reanimated** — перетягування з обмеженням країв, зум і повернення до ТОЧКИ
- **Нуль нових залежностей** — `gesture-handler` і `Reanimated` уже були в проєкті



Заміряно, а не на віру

–62%

рендерів ProductCard
при наборі в пошуку: 42 → 16

–50%

рендерів CartItem
при зміні кількості: 10 → 5

–3,5 МіБ

шрифтів іконок
19 файлів → 1

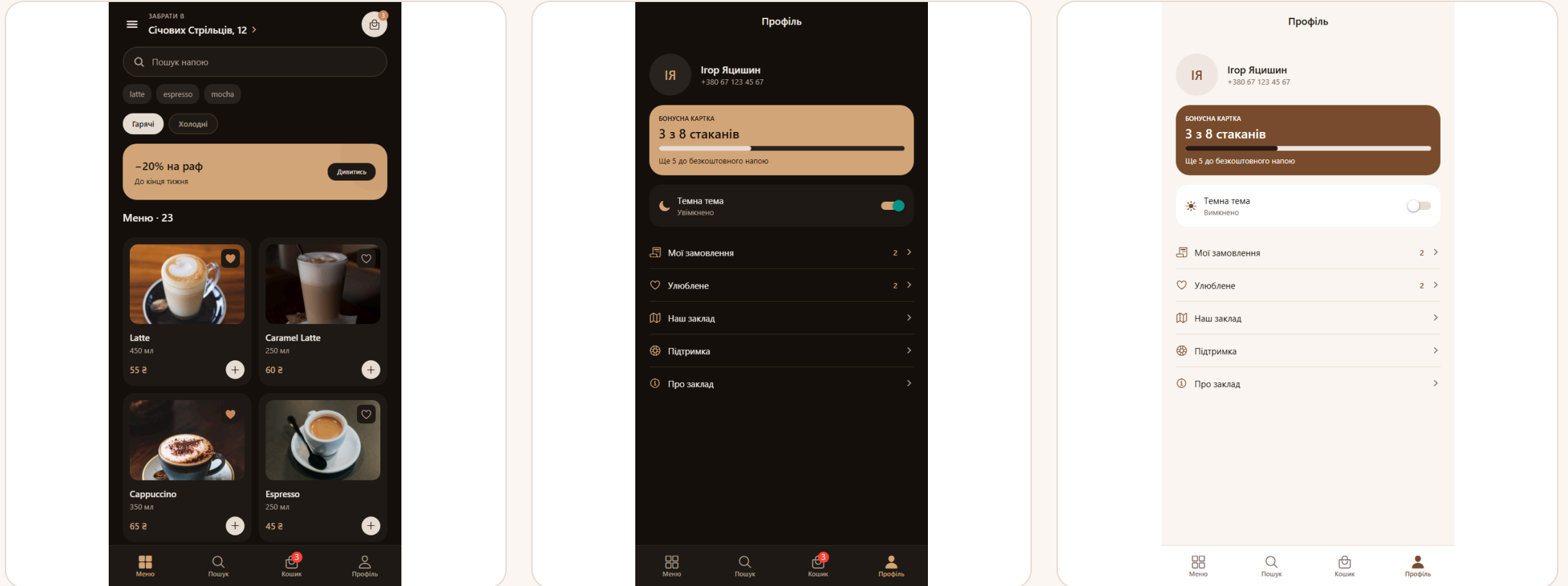
- **React.memo працює тільки в парі з useCallback** — доки пропси-функції створюються заново, порівняння завжди провалюється
- **FavoriteButton підписується на стор сам** — інакше кожне сердечко оновлювало б пропси всієї сітки
- **SearchBar свідомо без memo** — його value змінюється на кожен символ, порівняння було б витратами без користі
- **Точковий імпорт іконок** — бочковий тягнув усі 15 наборів із гліф-мапами та шрифтами

Як організований код

api/	запити й мапер відповіді
components/	UI, кожен в окремому файлі
context/	тема через Context API
data/	підказки пошуку, дані закладу
dev/	лічильник рендерів (вимкнений)
hooks/	useRequest і обгортки над ним
navigation/	навігатори, маршрути, deep links
screens/	екрани застосунку
storage/	читання й запис стану
store/	слайси, стор, middleware
theme/	палітри, типографіка, метрики
utils/	дати, математика тайлів

- **Кожен компонент — окремий файл** — дані приходять пропсами
- **Жодних магічних чисел** — кольори, відступи, радіуси й межі винесені в константи
- **Маршрути тільки через константи** — `SCREENS`, `STACKS`, `DRAWER`
- **Спільний життєвий цикл запиту** — `useRequest`, а не копія на кожен екран
- **Категорія — частина ідентичності напою** — `/hot` і `/iced` перевикористовують id

Дві теми, один код



Стилі оголошені як фабрики `createStyles(colors): StyleSheet.create` зафіксував би кольори однієї теми назавжди. Компонент не знає, яка тема активна.

ПІДСУМОК

Що змінилось

- **Наскрізний флоу замовлення** — від кошика до історії, зі збереженням
- **Три нові екрани** — улюблене, заклад із картою, оновлена історія
- **Чотири слайси стану** — кошик, замовлення, улюблене плюс тема в Context
- **Нічого не втрачається** — перезапуск повертає кошик, замовлення, улюблене й тему
- **Два виправлені баги** — зіткнення id між категоріями та перезапис збереженої теми